技术部第三次技术分享会

数组、方法与面向对象

姚雨森　技术部分管主席

三件商品：用变量保存价格

购买三件商品，价格分别为 12 元、8 元和 25 元，计算总价。

下面的代码写在 main 中：

```
int price1 = 12;
int price2 = 8;
int price3 = 25;

int total = price1 + price2 + price3;
System.out.println(total);
```

- 每件商品的价格保存在一个变量中。
- 把三个价格相加，输出总价 45。

商品变成三十件，怎么办

现在有 **30 件商品**，需要计算总价，并找出价格低于 10 元的商品。

- 保存价格：要从 `price1` 一直声明到 `price30`。
- 计算总价：要把 30 个变量逐个写进求和表达式。
- 筛选商品：要对每个价格重复写判断语句。

计算时，我们并不关心每个价格叫 `price1` 还是 `price2`，只需要对一组同类型的数据进行批量计算。

我们需要把这些价格放在一起，并通过编号访问，这就要用到**数组**。

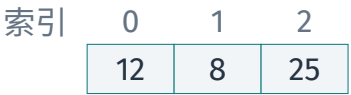
数组的声明与初始化：两种写法

`int[]` 表示整数数组类型，`prices` 是数组变量名。

给出初始元素

```
int[] prices = {12, 8, 25};
```

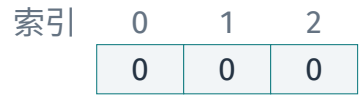
由初始元素个数确定长度。
这里有 3 个元素。



指定数组长度

```
int[] prices = new int[3];
```

方括号中的 3 指定长度。
整数数组的元素默认是 0。



每个格子代表一个元素；索引从 0 开始，三个元素的索引是 0、1、2。

通过索引读取和修改数组元素

使用**数组名 [索引]** 访问指定元素；`prices[1]` 表示第二个元素。

```
int[] prices = {12, 8, 25};
System.out.println(prices[1]);  // 8
int savedPrice = prices[1];
prices[1] = 10;
System.out.println(prices[1]);  // 10
System.out.println(savedPrice); // 8
```

- **读取**：可以直接打印元素，也可以用它参与计算。
- **保存**：`int savedPrice = prices[1];` 把当前的值存入新变量。
- **修改**：`prices[1] = 10;` 把第二个元素改为 10。

`savedPrice` 保存的是当时复制的整数值，不会随数组元素的修改而改变。

用 for 循环计算总价

遍历三个价格，把每个价格累加到总价中。

```
int[] prices = {12, 8, 25};
int total = 0;
for (int i = 0; i < 3; i++) {
    total += prices[i];
}
System.out.println(total);
```

- i 依次为 0、1、2，prices[i] 依次取出 12、8、25。
- 每轮执行 total += prices[i];，累加当前价格。
- 循环结束后，打印总价 45。

用 length 获取数组长度

`prices.length` 表示数组中元素的个数，这里是 3。

把循环条件中的 `i < 3` 改为 `i < prices.length`：

```
int[] prices = {12, 8, 25};
int total = 0;
for (int i = 0; i < prices.length; i++) {
    total += prices[i];
}
System.out.println(total);
```

- `length` 后面没有括号。
- 索引从 0 到 `length - 1`，所以循环条件使用 `<`。
- 增减初始化列表中的价格时，不需要再修改循环条件。

同一个公式，要重复写几遍

给定 double 类型的 x，计算并打印 $x^2 + 4x + 10$。

```
double x1 = 1.0;
double x2 = 2.0;
double x3 = 3.0;
System.out.println(x1 * x1 + 4 * x1 + 10);
System.out.println(x2 * x2 + 4 * x2 + 10);
System.out.println(x3 * x3 + 4 * x3 + 10);
```

- 输入不同，计算过程相同，却要重复书写同一个公式。
- 如果公式中的 `+ 10` 改为 `+ 11`，每一处都要修改，容易遗漏。

可以把计算过程写成函数，需要时调用；在 Java 中，函数通常称为**方法**。

定义方法，把计算过程写一次

这里先使用 static 方法，定义形式为：
static 返回类型方法名 (参数列表) { 方法体 }
方法定义在类中，与 main 并列：

```
static double calculate(double x) {
    double result = x * x + 4 * x + 10;
    return result;
}
```

在 main 中调用，并打印返回的结果：

```
System.out.println(calculate(1.0)); // 15.0
System.out.println(calculate(2.0)); // 22.0
System.out.println(calculate(3.0)); // 31.0
```

参数 x 接收输入；return 返回一个 double 类型的结果。

方法定义：返回类型、名字和参数列表

static 返回类型方法名 (参数列表) { 方法体 }

```
static double calculate(double x) {
    double result = x * x + 4 * x + 10;
    return result;
}
```

- **返回类型**：double 说明返回结果的类型，不是结果本身。
- **方法名**：calculate 是调用这个方法时使用的名字。
- **参数列表**：每个参数写成“类型名字”，多个参数用逗号分隔；没有参数时仍保留 ( )。

先把计算结果存入 result，再用 return result;结束当前方法并返回结果。返回的值必须与声明的返回类型兼容。

调用方法：传入参数，使用返回值

调用形式：**方法名 (实际参数)**，参数按定义中的顺序传入。

下面的代码写在同一个类的 main 中：

```
double result = calculate(2.0);
System.out.println(result);           // 22.0
System.out.println(calculate(3.0));  // 31.0
double total = calculate(1.0) + calculate(2.0);
System.out.println(total);            // 37.0
```

- calculate(2.0) 将 2.0 传给参数 x，然后执行方法体。
- 执行 return 后回到调用处，用返回的值继续计算或赋值。
- 调用时不写参数类型；参数的数量、顺序和类型要与定义匹配。

完整程序：调用后返回原处继续执行

保存为 Main.java，程序从 main 开始执行。

```
public class Main {
  public static void main(String[] args) {
    double value = calculate(2.0);
    System.out.println(value);
  }

  static double calculate(double x) {
    double result = x * x + 4 * x + 10;
    return result;
  }
}
```

执行顺序

1. 进入 main。
2. 调用 calculate，传入 2.0。
3. 计算得到 result = 22.0。
4. 返回 main，将结果赋给 value。
5. 继续打印 22.0。

参数按位置对应，顺序不能随意交换

定义一个接收两个参数的方法：

```
static double sum(int a, double b) {
    double result = a + b;
    return result;
}
```

在 main 中调用：

```
double total = sum(3, 2.5);
System.out.println(total); // 5.5
```

- 第一个实参 3 传给 int a，第二个实参 2.5 传给 double b。
- 实参按位置对应形参，不会根据类型自动调整顺序。
- sum(2.5, 3) 无法编译：2.5 不能直接传给 int a。

不同方法可以有同名的局部变量

下面两个方法并列定义在同一个类中：

```java
static double first() {
    double value = 10.0;
    return value;
}
```

```java
static double second() {
    double value = 20.0;
    return value;
}
```

在 main 中依次调用，两个方法都能正常运行：

```java
System.out.println(first());  // 10.0
System.out.println(second()); // 20.0
```

- **作用域**是可以通过变量名访问这个变量的代码范围。
- 两个 value 是各自方法中的局部变量，名字相同也互不冲突。
- main 不能直接访问它们，但可以接收方法返回的值。

方法也可以调用另一个方法

同一个类中，calculate 调用 square 来计算平方。

```
static double calculate(double x) {
    double squared = square(x);
    double result = squared + 4 * x + 10;
    return result;
}

static double square(double x) {
    double result = x * x;
    return result;
}
```

调用与返回顺序

1. calculate 接收 x = 2.0。
2. 调用 square(x)，传入 2.0。
3. square 返回 4.0，存入 squared。
4. 回到 calculate，继续计算并返回 22.0。

每次调用都会进入被调用的方法，返回后从本次调用的位置继续执行。

抽象：知道怎么用，不必知道怎么实现

调用一个方法，需要了解它的输入、返回结果和功能约定。

以 `double sum(int a, double b)` 为例，约定它返回两个参数的和：

- **传入什么**：先传一个 `int`，再传一个 `double`。
- **得到什么**：得到一个 `double` 类型的结果，值为两者之和。
- **如何使用**：调用方法，再保存、打印或继续使用返回值。

```
double total = sum(3, 2.5);
System.out.println(total); // 5.5
```

调用者不需要知道内部变量叫什么、计算分几步完成。只要实现遵守约定，就可以通过调用获得预期结果。

一件商品，需要保存多种信息

现在除了价格，还要记录每件商品的名称和库存。

可以先用三个数组分别保存：

```
String[] names = {"Notebook", "Pen", "Eraser"};
double[] prices = {12.0, 8.0, 3.0};
int[] stocks = {10, 20, 30};
```

- 索引 0：Notebook，价格 12 元，库存 10 件。
- 索引 1：Pen，价格 8 元，库存 20 件。
- 索引 2：Eraser，价格 3 元，库存 30 件。

我们约定：三个数组中相同索引的数据，属于同一件商品。

只交换名称，商品信息就对应错了

想把 Pen 排在 Notebook 前面，却只交换了名称：

```
String temp = names[0];
names[0] = names[1];
names[1] = temp;
```

索引	名称	价格（元）	库存（件）
0	Pen	12.0	10
1	Notebook	8.0	20
2	Eraser	3.0	30

Pen 被配上了 Notebook 的价格和库存，但程序不会因此报错。

调整顺序时，三个数组必须同步修改；商品信息之间的对应关系，全靠我们维护。

用类描述商品，把相关数据组织在一起

名称、价格和库存，本来就在描述同一件商品。我们希望移动一件商品时，这些信息也能作为一个整体移动。

用 `Product` 类描述：**每件商品都有名称、价格和库存**。

一个商品对象

名称：`Notebook`
价格：12 元
库存：10 件

另一个商品对象

名称：`Pen`
价格：8 元
库存：20 件

类描述共同的结构；**对象（实例）**保存某一件具体商品的数据。

定义类：把相关的量写在一起

使用 `class 类名 { ... }` 定义一个类。将下面的代码保存为 `Product.java`：

```
class Product {
    String name;
    double price;
    int stock;
}
```

- `Product` 是类名，也可以作为变量的类型。
- 类中声明的这些量称为**字段（成员变量）**。
- 每个 `Product` 对象都有自己的 `name`、`price` 和 `stock`。

这段定义描述商品有哪些数据；具体商品的数据要保存在创建出的对象中。

用商品对象替代三个分散的数组

下面的代码写在同一目录下 Main.java 的 main 中：

```
Product notebook = new Product();
notebook.name = "Notebook";
notebook.price = 12.0;
notebook.stock = 10;
Product pen = new Product();
pen.name = "Pen";
pen.price = 8.0;
pen.stock = 20;
Product eraser = new Product();
eraser.name = "Eraser";
eraser.price = 3.0;
eraser.stock = 30;
```

- new Product() 创建一个商品对象。
- 用“**对象变量.字段名**”访问或修改这个对象的数据。
- 三件商品分别保存各自的名称、价格和库存。

调整商品顺序，只需交换对象

接着把三个商品对象放进一个 Product[] 数组：

```
Product[] products = {notebook, pen, eraser};

Product temp = products[0];
products[0] = products[1];
products[1] = temp;
```

此时，索引 0 对应的是 pen，它仍保存自己的价格和库存：

```
System.out.println(products[0].name);  // Pen
System.out.println(products[0].price); // 8.0
System.out.println(products[0].stock); // 20
```

每个数组元素对应一个完整的商品对象，不必再同步交换三个数组中的数据。

构造方法：创建对象时就把数据填好

把字段赋值写进**构造方法**，创建对象时直接传入初始数据。

```
class Product {
    String name;
    double price;
    int stock;
    Product(String initialName, double initialPrice,
            int initialStock) {
        name = initialName;
        price = initialPrice;
        stock = initialStock;
    }
}
```

- 构造方法与类同名，不写返回类型，也不写 `void`。
- 执行 `new Product(...)` 时，会调用匹配的构造方法。

创建时传入参数，构造方法完成初始化

在 main 中创建商品，不再逐条给字段赋值：

```
Product notebook = new Product("Notebook", 12.0, 10);
Product pen = new Product("Pen", 8.0, 20);
Product eraser = new Product("Eraser", 3.0, 30);
```

参数按顺序传给构造方法，创建出的对象已经保存名称、价格和库存。

构造方法里也可以执行其他语句，例如在字段赋值后打印创建提示：

```
// At the end of the constructor:
System.out.println("Created: " + name);
```

每次通过这个构造方法创建商品，都会执行这条提示语句。

构造方法用于完成对象的初始化，可以赋值，也可以进行计算或条件判断。

对象也可以作为方法参数

用类名作为参数类型，让方法接收一个商品对象。在 Main 类中定义：

```
static double stockValue(Product product) {
    double result = product.price * product.stock;
    return result;
}
```

在 main 中创建商品，并把它传给方法：

```
Product pen = new Product("Pen", 8.0, 20);
double value = stockValue(pen);
System.out.println(value); // 160.0
```

- 参数 product 的类型是 Product，调用时传入对象变量 pen。
- 方法通过 product.price 和 product.stock 读取这件商品的数据。
- 库存总价值 = 单价 $\times$ 库存数量，这里是 $8 \times 20 = 160$ 元。

把商品的数据和相关操作放在一起

商品已经保存了价格和库存，计算库存总价值也是与商品有关的操作。

如果相关操作都放在其他类中，每次调用都需要把商品传进去：

```
double value = stockValue(pen);
```

可以把这个操作定义在 Product 类中，让商品对象提供它：

```
double value = pen.stockValue();
```

- **字段**保存商品的数据，**实例方法**提供处理这些数据的操作。
- pen.stockValue() 表示计算 pen 这件商品的库存总价值。
- 数据与相关操作归在同一个类中，便于查找、使用和维护。

完整定义：Product.java

```
class Product {
  String name;
  double price;
  int stock;
  Product(String productName, double unitPrice, int quantity) {
    name = productName;
    price = unitPrice;
    stock = quantity;
  }
  double stockValue() {
    double result = price * stock;
    return result;
  }
}
```

完整调用：Main.java

两个文件放在同一目录，main 通过具体对象调用实例方法。

```
public class Main {
  public static void main(String[] args) {
    Product pen = new Product("Pen", 8.0, 20);
    Product notebook = new Product("Notebook", 12.0, 10);
    System.out.println(pen.stockValue());      // 160.0
    System.out.println(notebook.stockValue()); // 120.0
  }
}
```

- 实例方法不写 static，也不再显式接收 Product 参数。
- 点号前的对象决定使用谁的字段；不同对象使用各自的价格和库存。

比较两件商品，方法里需要区分谁是谁

给 Product 增加一个比较单价的实例方法，期望这样调用：

```
Product pen = new Product("Pen", 8.0, 20);
Product notebook = new Product("Notebook", 12.0, 10);
boolean cheaper = pen.isCheaperThan(notebook);
System.out.println(cheaper); // true
```

这次调用表示：**这支笔是否比这本笔记本便宜？**

- 点号前的 pen：当前调用方法的对象，单价为 8 元。
- 括号中的 notebook：传入的另一个对象，单价为 12 元。
- 方法需要分别读取当前对象和另一个对象的价格。

在实例方法内部，可以用 this 表示**当前对象**。

this 表示当前调用方法的对象

在 Product 类中加入这个实例方法：

```
boolean isCheaperThan(Product other) {
    boolean result = this.price < other.price;
    return result;
}
```

- 调用 pen.isCheaperThan(notebook) 时，this 指向 pen，other 指向 notebook，比较的是 $8 < 12$。
- 交换调用对象，this 和参数所指的对象也随之改变：

```
boolean cheaper = notebook.isCheaperThan(pen);
System.out.println(cheaper); // false: 12.0 < 8.0
```

stockValue() 中的 price * stock 也可写为 this.price * this.stock。

this 区分字段与同名参数

构造方法的参数可以与字段同名，用 this 明确访问当前对象的字段：

```
Product(String name, double price, int stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
}
```

- 左侧 this.price 是当前商品对象的字段。
- 右侧 price 是构造方法接收到的参数。
- this.price = price; 把参数的值保存到当前对象中。

如果写成 price = price;，两边都指参数，只是给参数自己赋值，不会给对象的字段赋值。

static 成员属于类，实例成员属于对象

实例成员：对应某个对象

- 每件商品有自己的价格、库存。
- 实例方法通过对象调用，内部有 this。

```
double p = pen.price;
double v = pen.stockValue();
```

静态成员：属于类

- 静态字段由同一个类的实例共享。
- 静态方法通过类名调用，内部没有 this。

```
int n = Product.createdCount;
Main.stockValue(pen);
```

例如，在 Product 类中声明 static int createdCount = 0;，在构造方法中执行 createdCount++;，统计累计创建的商品对象数量。

静态方法需要对象数据时，可以通过参数显式传入对象。

静态字段：为每件商品分配编号

下面只保留编号相关代码，演示每创建一个商品就分配一个新编号：

```
class Product {
  static int nextId = 1;
  int id;
  Product() {
    id = nextId;
    nextId++;
  }
}
```

在 main 中依次创建：

```
Product a = new Product();
Product b = new Product();
System.out.println(a.id); // 1
System.out.println(b.id); // 2
```

此时 Product.nextId 为 3，留给下一个新对象。

- static int nextId：所有商品共享同一个计数器，不会随每次创建重置。
- int id：每个商品各自保存编号，计数器增加不会改变已有编号。
- 若 nextId 不加 static，每个新对象都会从 1 开始，编号就重复了。

直接修改字段，可以绕过业务规则

商品库存应该大于或等于 0，但目前外部可以直接赋值：

```
Product pen = new Product("Pen", 8.0, 20);
pen.stock = -10;
System.out.println(pen.stock); // -10
```

- -10 是合法的 int，这条赋值语句不会因为库存为负而报错。
- 即使另写了检查库存的方法，外部仍可以绕过它直接修改字段。
- 我们需要限制直接访问，让修改经过统一的检查入口。

访问权限修饰符用来控制哪些代码可以访问字段、方法等成员。

private：限制类外直接访问字段

将 Product 的库存字段声明改为：

```
private int stock;
```

Product 类内部仍能访问它，例如构造方法中的赋值：

```
this.stock = stock;
```

但在 Main 中，下面两条语句都会编译失败：

```
pen.stock = -10;
System.out.println(pen.stock);
```

public：通过公开方法修改库存

在 Product 中提供 public 方法，允许外部调用；库存字段保持私有。

```
public boolean purchase(int quantity) {
    if (quantity <= 0 || quantity > stock) {
        return false;
    }
    stock -= quantity;
    return true;
}
```

在 main 中调用，假设 pen 当前库存为 20：

```
boolean success = pen.purchase(2);
System.out.println(success); // true
```

成功后库存从 20 变为 18；数量不合法或库存不足时返回 false，库存不变。

getter 和 setter：读取与设置字段

在 Product 类中，为私有字段 stock 提供方法：

getter：返回字段的值

```
public int getStock() {
    return stock;
}
```

setter：检查后更新字段

```
public void setStock(int stock) {
    if (stock >= 0) {
        this.stock = stock;
    }
}
```

```
pen.setStock(20);
System.out.println(pen.getStock()); // 20
```

- void 表示不返回值；这里负数输入被忽略。
- getter、setter 是普通方法的命名约定，不必为每个字段提供 setter。